Carnegie Mellon University

95-891

Introduction to Artificial Intelligence

Insider Threat Detection

04/23/2026

Seth Payson
Luthfan Akbar
Brendan Reed

Agenda

Executive Summary

Problem Overview

Data

Methodology

Results

Conclusions

Q&A

Executive Summary

- **Goal:** Detect insider threats using behavioral data
- **EDA:** Aggregated logs into user-day features
- **Approach:** Semi-supervised - LSTM Autoencoder
  - Additional stacked model approach
- **Key Findings:** Behavioral anomaly detection can be useful but know the limitations

The Problem

The Insider Threat Problem

- Insider threats are hard to detect
- Malicious users resemble normal users
- Behavior can be subtle
- Traditional tools focus on external threats

The average annual cost of insider security incidents
$19.5 million

Source: Ponemon - 2026 Cost of Insider Risks Global Report (https://ponemon.dtex.ai/#download)

Goal

Detect anomalous user behavior patterns.

Develop a model that works on “needle in a haystack” problems with low-likelihood, high-impact scenarios.

Data

Input - CERT Insider Threat Dataset (v6.2)

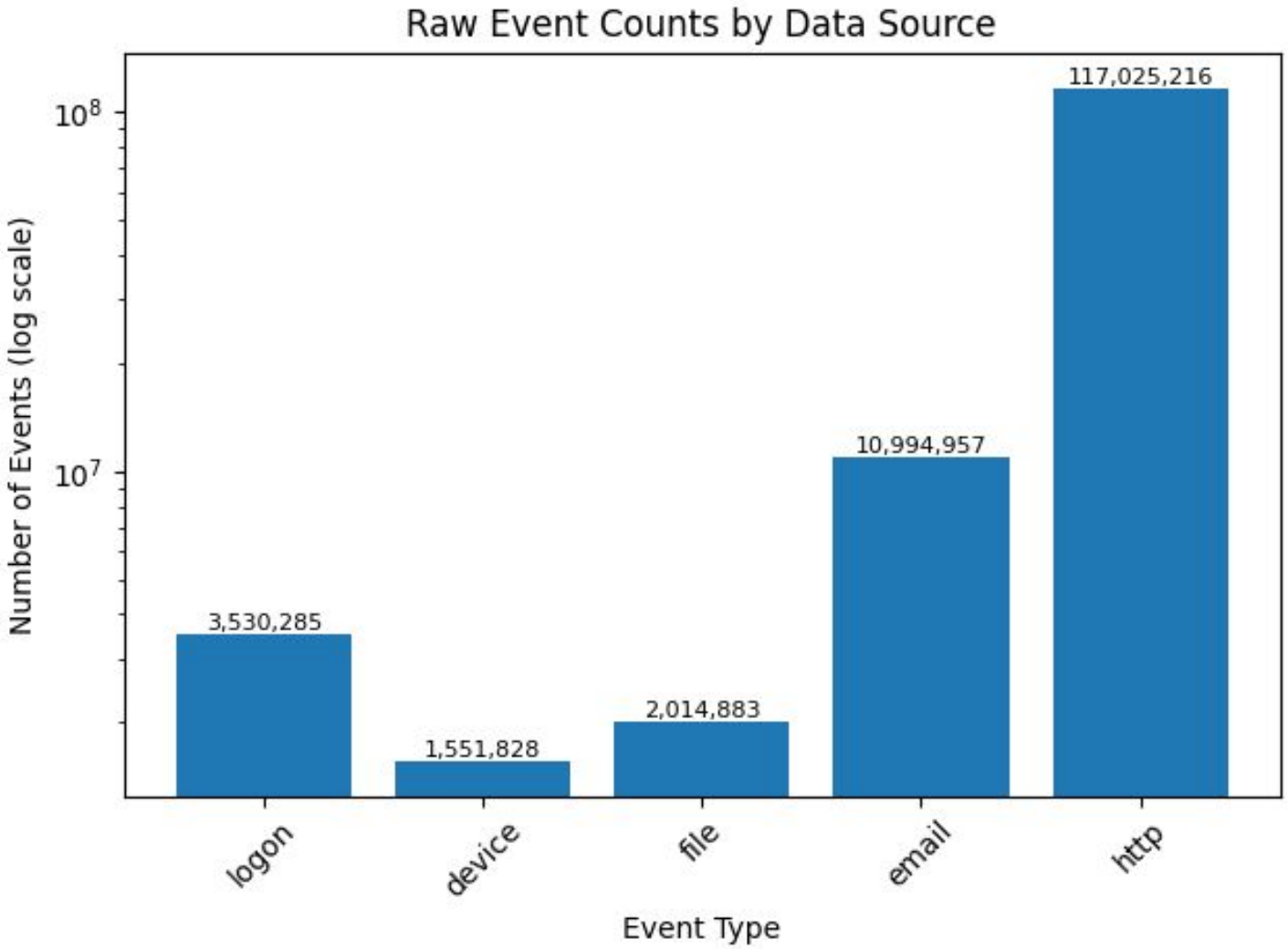
Before Aggregation

- Raw logs: logon, file, device, email, http
- High volume, event level data
- Difficult to interpret behavior

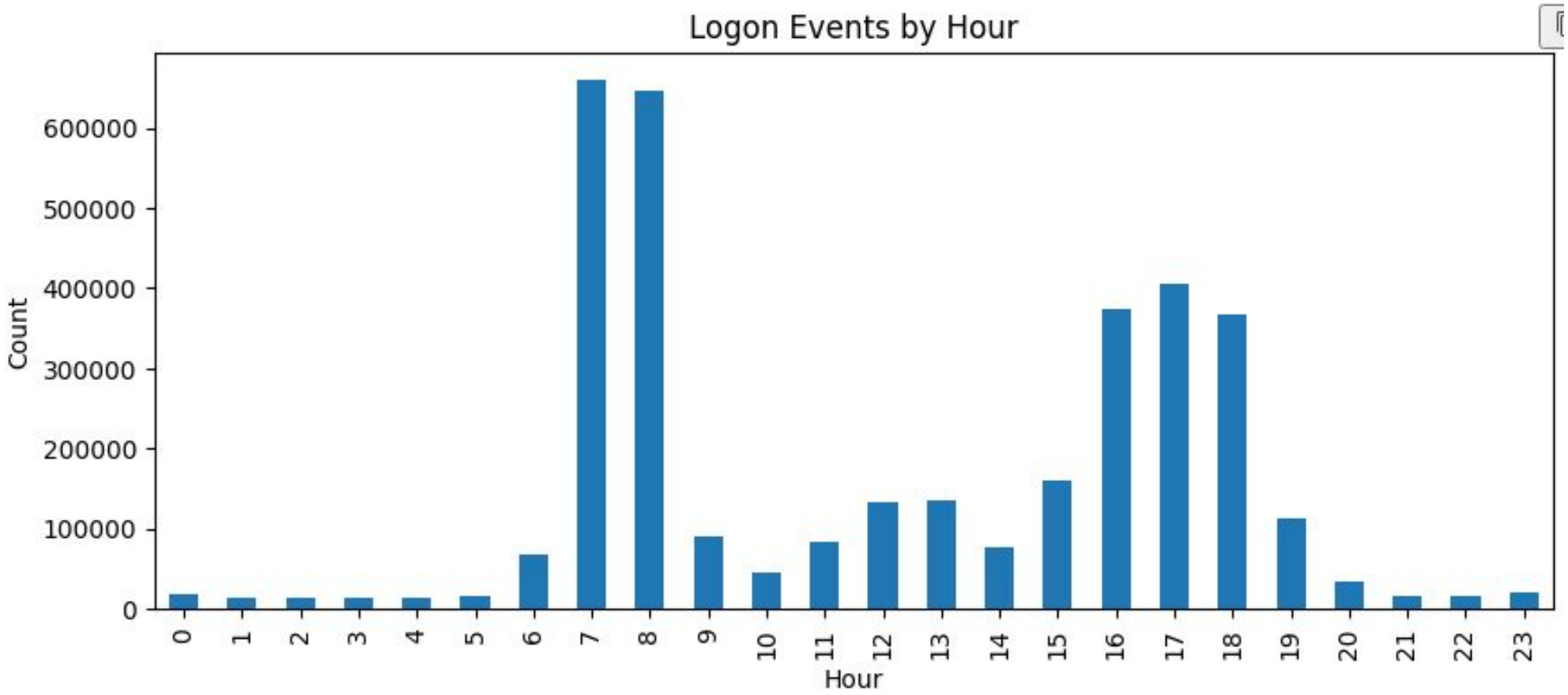
After Aggregation

- User-day dataset
- Behavorial features (counts, patterns, timing)
- Enables anomaly detection based on behavior

Before Aggregation



After Aggregation



Aggregated User-Day Dataset Overview

Full data set

- 4000 users
- ~1.3 million user-day rows
- 5 total malicious users
- 51 total features

Test / Train Split

- 80/20 split on benign users
- 5 malicious users assigned to test set

- Train: 3196 users
- Test: 804 users

Carnegie Mellon University

Example User-Day Row

user_id = AEDY1738

date = 3.11.2011

logon_count = 3

after_hours_events = 1

file_copy_count = 12

http_upload_count = 0

external_email_count = 2

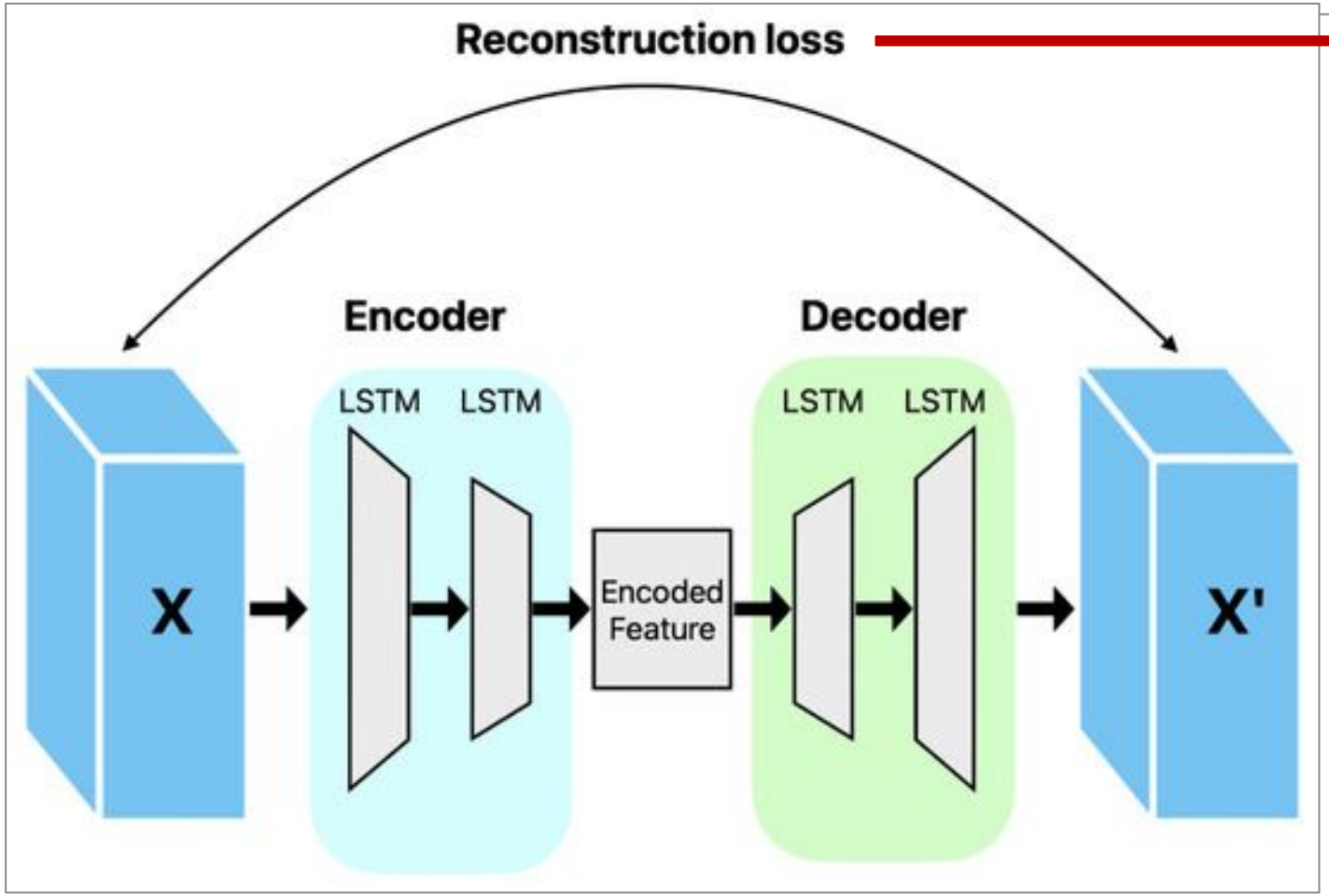
Methodology

The Case for Unsupervised Anomaly Detection using LSTM AUTOENCODER

1. User behavior is temporal, where threat patterns unfold over days, not single events.
2. Supervised models only recognize known attack patterns. Unsupervised reconstruction flags any deviation from normal, including novel tactics.
3. Eliminates concerns of inadequate training/scaling of supervised methods.
4. Normal behavior is well-defined and deviations from baseline are the key signal.

Encoder-Decoder Architecture

Importance of Reconstruction Error



MSE Loss = Anomaly Score

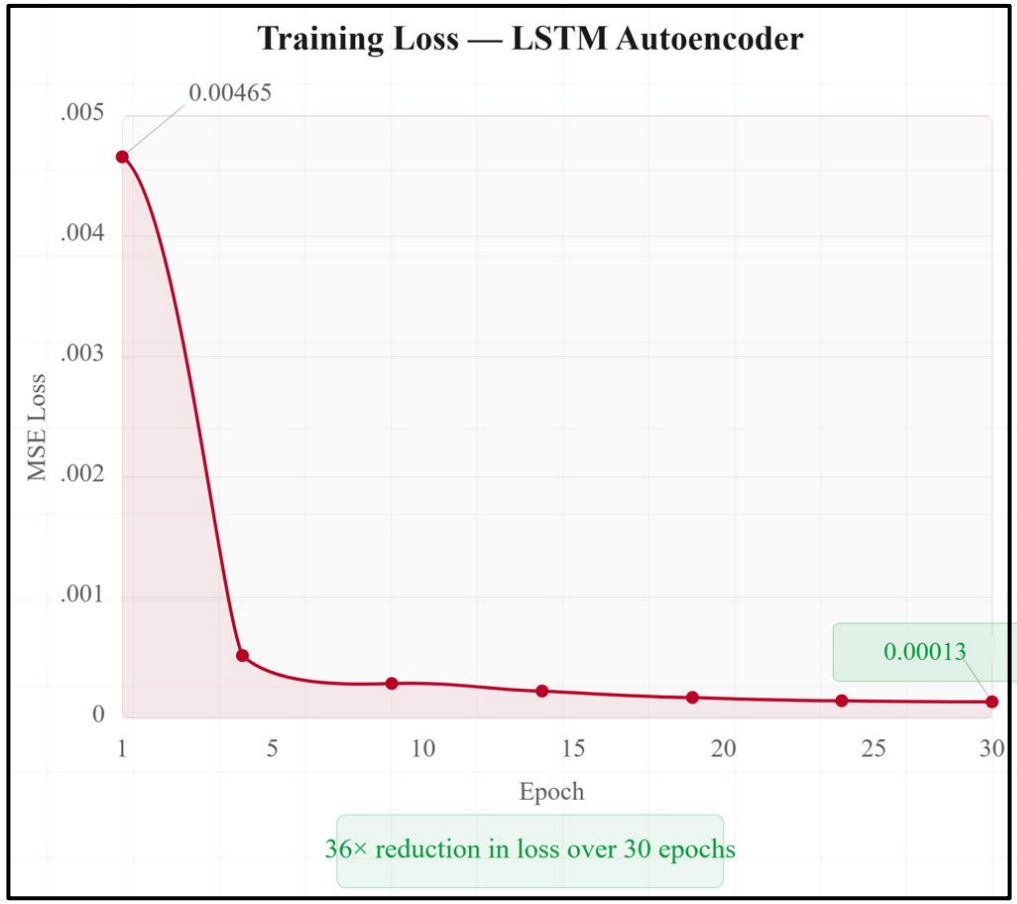
Our model was adapted from a publicly available codebase using an autoencoder to detect heart defects by tracking anomalies in echocardiogram results.

Model Training - Baseline of Benign Sequences Only

- A 5-day sliding window is used to chunk chronological data digestible by the model.

- Stride = 5 (training): reduces input size.
- Stride = 1 (test): dense window to maximize coverage)

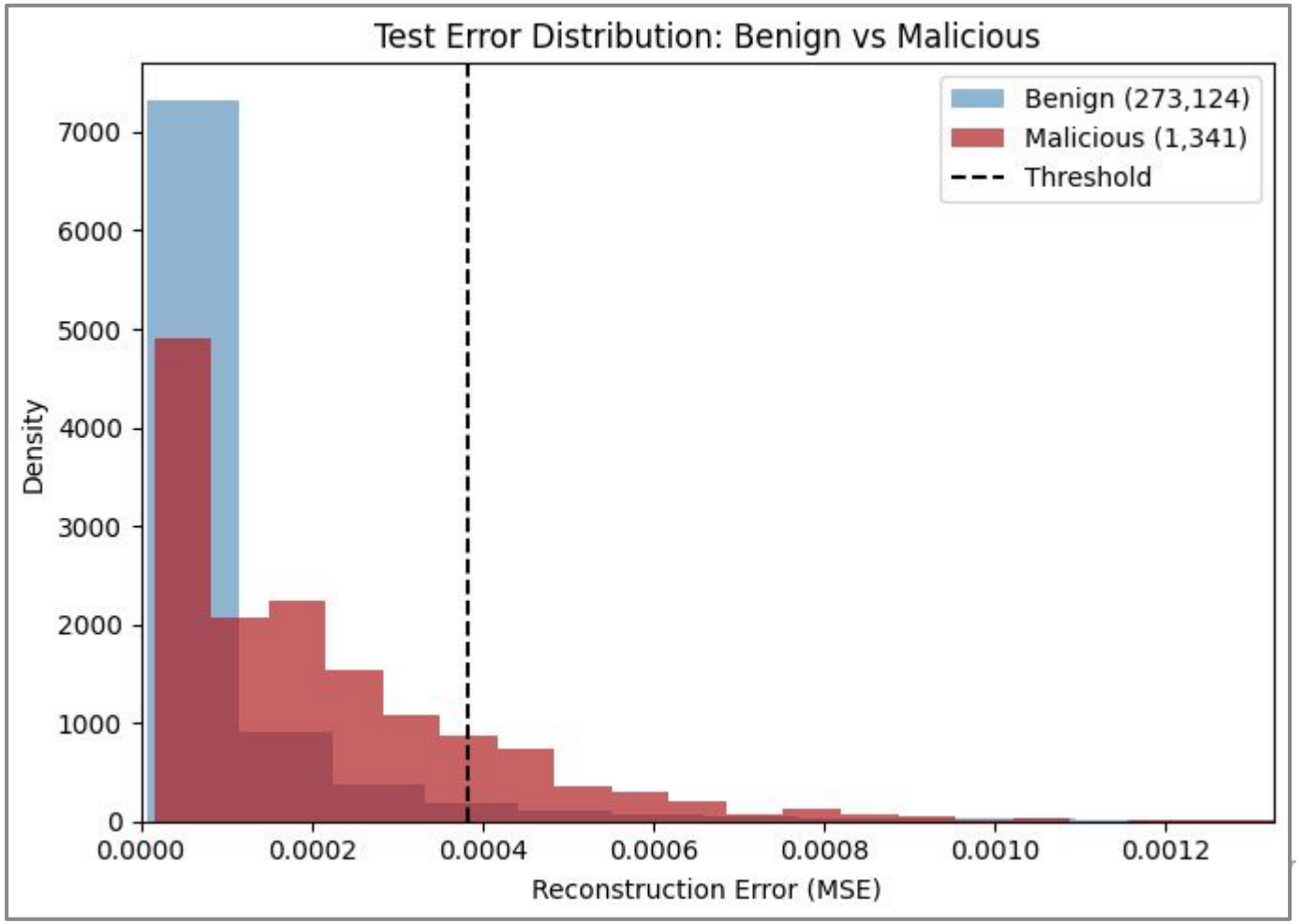
- **Zero malicious windows in training data**



Reconstruction Error as Anomalous Signal

An LSTM autoencoder trained on normal time series data will encode data efficiently in its inner representations.

However, when anomalous data is fed to the network, the decoder will not be able to reconstruct this data properly since the encoder did not see such patterns during training. The higher reconstruction error will indicate the presence of an anomaly.



Overall Model Performance:

5 / 5

Malicious users detected

100% recall at user level
zero missed threats

213

False-positive users

Anomalous but benign — passed to Stage 2 XGBoost for disambiguation

218

Total flagged users

Input to supervised stage; all 5 malicious users included

73%

Reduction in user count

From 804 original users, only 26% flagged with mild confidence of malicious action

Overall Model Performance:

Metric	Value	Notes
ROC-AUC	0.7761	Independent of threshold level; measures overall ranking quality
PR-AUC (Avg. Precision)	0.0135	Expected to be low given 0.49% positive rate
Accuracy	0.94	This is a misleading value: baseline "predicting all benign" results in this high score
Precision (malicious)	0.02	2 in every 100 flagged windows are truly malicious. Expected to be low.
Recall (malicious)	0.18	18% of malicious **rows** individually flagged
F1 (malicious)	0.03	Reflects the precision/recall imbalance at **row** level
User-level Recall	1.00	All 5 malicious users flagged
User-level Precision	0.016	5 true positives out of 319 flagged users

Surprises and late-stage improvements:

- **Sliding window length** had a major impact on recal than any model parameter. Originally choosing a window and stride of 30 days captured only 3 of 5 malicious users and produced 94 false positives. Reducing the window to 5 days allowed the model to detect shorter-duration behavioral anomalies and recovered all 5 malicious users.
- **The top 10 users by maximum reconstruction error were all false positives:** The highest reconstruction errors in the test set belonged to benign users, not malicious ones.

UserID: HGH0727	Reconstruction Error:	0.00653	(Benign)
UserID: CDE1846	Reconstruction Error:	0.004034	(Malicious)

Next steps: Two Paths Forward:

1. **Human Analyst Review:**
   Results from the autoencoder reduce our employee list from >800 to 218 users. This list is now compact enough to be adequately triaged by a human analyst for further investigation.

2. **Supervised Model Refinement:**
   Feed the 218 flagged users into a supervised classifier (XGBoost) to distinguish anomalous-but-benign from anomalous-and-malicious.

   The LST reconstruction error becomes an additional feature.

Model Selection for Stage 2: Supervised Classification

Why do we need a second stage?

- LSTM flagged 222 suspicious users — but most are false positives
- We need a model that can separate the truly malicious from anomalous-but-innocent
- LSTM gave us a shortlist. XGBoost gives us a verdict

Why XGBoost?

- Handles severe class imbalance natively — no synthetic data needed
- Fast enough to train on the flagged pool in Colab
- Outperformed Random Forest (0.966 vs 0.950 AUC-ROC)

Input and Output

LSTM Input (LSTM test dataset)

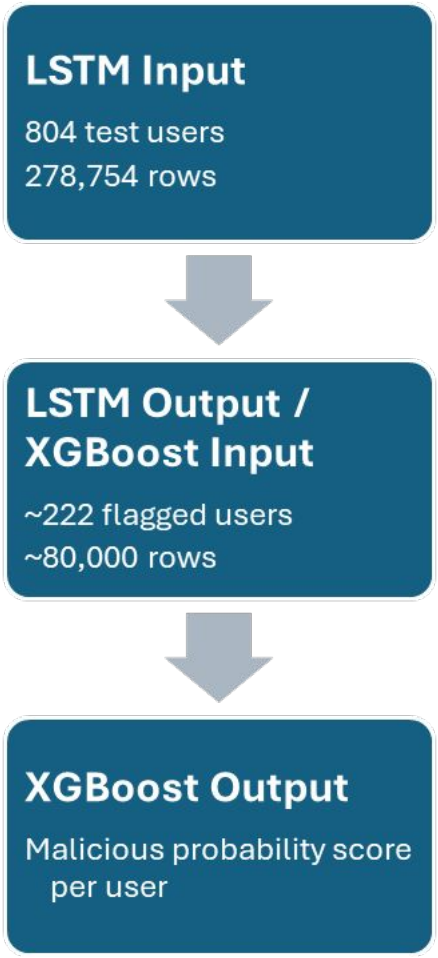
- 804 test users
- 278,754 user-day rows
- 5-day behavioral windows across 44 features

LSTM Output / XGBoost Input:

- ~222 flagged users · ~80,000 rows
- Behavioral features + 3 LSTM reconstruction error scores

XGBoost Output:

- Malicious probability score per user-day
- Risk-ranked list for security analyst review



Design and Strategy

Model Design:

- Trained only on LSTM-flagged anomalous days; not the full dataset
- Class imbalance handled via *scale_pos_weight*; no synthetic data needed
- LSTM reconstruction error included as an extra feature
- Split by user not by row; prevents data leakage

Evaluation Strategy

- Train on 3 malicious users, test on 2 unseen malicious users
- Primary metric: AUC-ROC
- Secondary: did we catch the right people, on the right day?

Results

LSTM + XGBoost Pipeline

- AUC-ROC: 0.969 on flagged anomalous days
- At threshold 0.85: 1 false positives
- Correctly identified the exact malicious day for 2/2 test malicious users with 0.999 confidence

Sample Risk Ranking

User	Score	Is Malicious	Model Correct?
CMP2946	0.9999	✓	✓
PLJ1771	0.9999	✓	✓
IIW0399	0.8534	✗	✗ False positive
Everyone else	< 0.55	✗	✓ Correctly benign

2 / 2

Malicious users detected

100% recall on unseen test users — zero missed threats

1

False positive at threshold 0.85

Threshold tuned on train set — 1 benign user incorrectly flagged

0.969

Stage 2 AUC-ROC

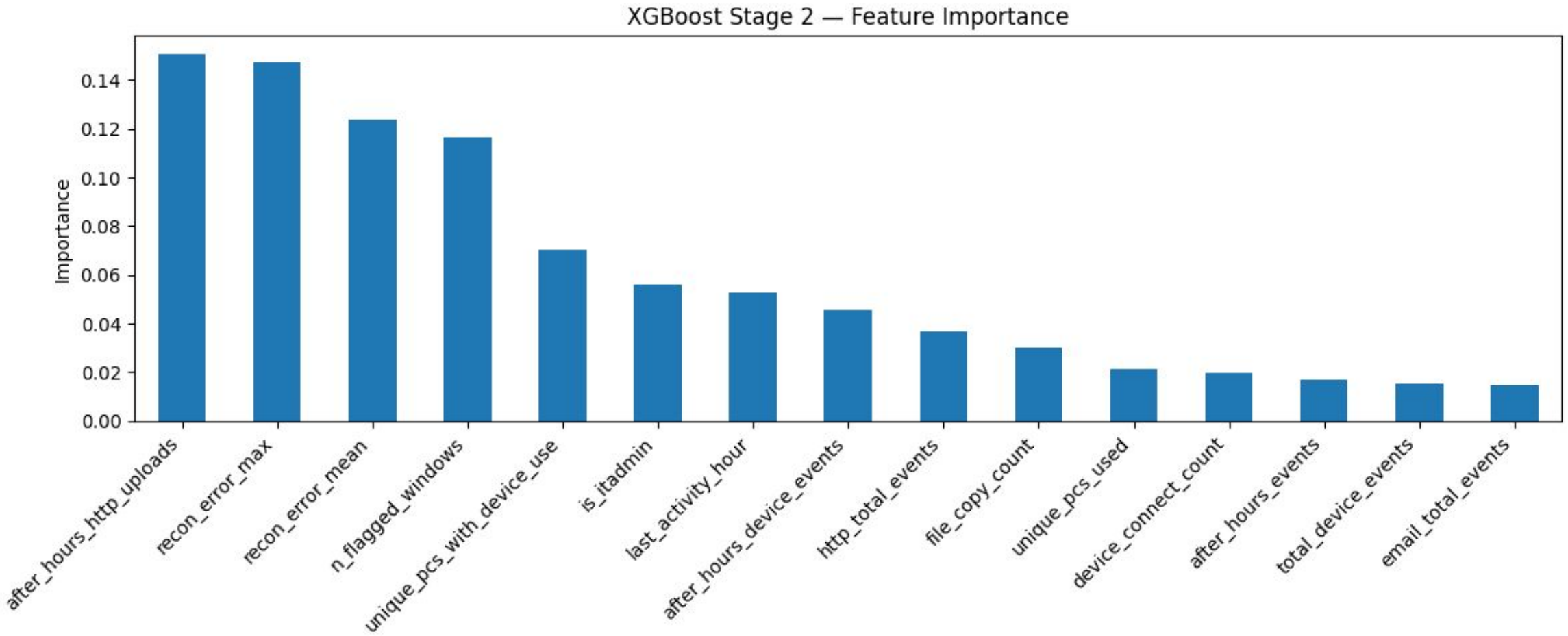
On LSTM-flagged anomalous days — strong discrimination

0.9999

Confidence on malicious users

Exact malicious day correctly identified at top of risk ranking

XGBOOST - STAGE 2



- **after_hours_http_uploads** is the top signal: malicious users exfiltrated data via web uploads outside working hours
- LSTM reconstruction error ranks 2nd and 3rd: confirms LSTM is useful input for XGBoost
- **n_flagged_windows** at 4th: users with more anomalous windows are more likely to be malicious

Limitations
Next Steps
Conclusion

Limitations

- User-day aggregation may miss short or isolated malicious events
- Model is better at detecting behavioral change than one-off actions
- Extreme class imbalance limits generalization and evaluation
- High false positives would increase analyst workload

Next Steps

- Use finer time windows to better capture short lived attacks
- Reduce false positives through threshold tuning
- Test on more modern datasets

Conclusion & Takeaways

- Behavioral aggregation was key for our models to detect anomalies
- Anomaly detection can be effective for rare threat scenarios
- User-level insights provide more value than event level predictions
- Models support investigation by prioritizing suspicious user behaviors

Q&A

Overall Model Performance:

Metric	Value	Notes
ROC-AUC	0.969	On LSTM-flagged anomalous days only
Recall (malicious)	0.95	At threshold 0.90
Precision (malicious)	1.00	0 false positives at threshold 0.90
User-level recall	1.00	2/2 test malicious users detected
User-level precision	0.053	2 true positives out of 38 flagged users
Top user score	0.9999	Malicious user ranked #1 with near-perfect confidence
Score gap		0.9999 (malicious) vs 0.770 (highest false positive) vs 0.064 (next benign)

Results/Effectiveness

Direct XGBoost — CV AUC-ROC: 0.966

- Strong cross-validation performance on training data
- Assigned near-zero malicious probability to all test users
- 0 out of 5 malicious users detected on held-out test set

Direct Random Forest — CV AUC-ROC: 0.950

- Similar story — strong on training, failed on test
- 0 out of 5 malicious users detected on held-out test set
- Slower to train with no benefit over XGBoost

Both direct supervised models achieved strong cross-validation scores but detected zero malicious users on the held-out test set

Lessons Learned

- Aggregation enabled behavior based detection.
- Temporal window choice significantly impacted performance.
- Class imbalance made evaluation challenging.
- Models are better for ranking users than making hard classifications.
- Data understanding was as important as model design.

Output

- Anomaly score for each user-day.
- Identification of suspicious or anomalous behavior.
- Flags for potential insider threat activity.

Defining Success - Industry

Notes:

Detecting insider threats through user behavior

Detecting anomolous time based behaviors

Why This Problem Matters

- Insider threats can lead to data breaches, financial loss, and reputational damage.
- Malicious insiders already have authorized access, making them harder to detect.
- Traditional security tools are more effective against external threats than internal ones.
- Organizations need better methods to detect suspicious behavior early.